

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего
образования
«Национальный исследовательский университет ИТМО»
Мегафакультет компьютерных технологий и управления
Факультет Программной инженерии и компьютерной техники

Основы программной инженерии

Лабораторная работа №4
Вариант: 2539

Работу выполнил:
Здор Матвей
Максимович
Группа: Р3224
Преподаватель:
Миху Вадим
Дмитриевич

Санкт-Петербург
2025

Содержание

1	Задание	2
2	Реализация MBean'ов	3
2.1	MBean для подсчёта общего числа установленных точек и числа точек вне области с отправкой оповещения при выходе за границы	3
2.2	MBean для определения среднего интервала между кликами пользователя	5
3	Мониторинг с помощью JConsole	6
3.1	Снятие показаний MBean-классов	7
3.2	Определение Java Language Specification	13
4	Мониторинг и профилирование с помощью VisualVM	14
4.1	Графики изменения показаний MBean-классов во времени	14
4.2	Определение класса с наибольшим использованием памяти	15
5	Локализация и устранение проблем производительности	16
6	Заключение и выводы	23

1 Задание

1. Для своей программы из лабораторной работы #3 по дисциплине "Веб-программирование" реализовать:

- MBeap, считающий общее число установленных пользователем точек, а также число точек, попадающих в область. В случае, если количество установленных пользователем точек стало кратно 10, разработанный MBeap должен отправлять оповещение об этом событии.
- MBeap, определяющий средний интервал между кликами пользователя по координатной плоскости.

2. С помощью утилиты JConsole провести мониторинг программы:

- Снять показания MBeap-классов, разработанных в ходе выполнения задания 1.
- Определить версию Java Language Specification, реализуемую данной средой исполнения.

3. С помощью утилиты VisualVM провести мониторинг и профилирование программы:

- Снять график изменения показаний MBeap-классов, разработанных в ходе выполнения задания 1, с течением времени.
- Определить имя класса, объекты которого занимают наибольший объем памяти JVM; определить пользовательский класс, в экземплярах которого находятся эти объекты.

4. С помощью утилиты VisualVM и профилировщика IDE NetBeans, Eclipse или Idea локализовать и устранить проблемы с производительностью в программе. По результатам локализации и устранения проблемы необходимо составить отчет, в котором должна содержаться следующая информация:

- Описание выявленной проблемы.
- Описание путей устранения выявленной проблемы.
- Подробное (со скриншотами) описание алгоритма действий, который позволил выявить и локализовать проблему.

Студент должен обеспечить возможность воспроизведения процесса поиска и локализации проблемы по требованию преподавателя.

Отчёт по работе должен содержать:

1. Текст задания.
2. Исходный код разработанных MBeap-классов и сопутствующих классов.
3. Скриншоты программы JConsole со снятыми показаниями, выводы по результатам мониторинга.
4. Скриншоты программы VisualVM со снятыми показаниями, выводы по результатам профилирования.
5. Скриншоты программы VisualVM с комментариями по ходу поиска утечки памяти.
6. Выводы по работе.

Рис. 1: Текст задания лабораторной работы

2 Реализация MBean'ов

2.1 MBean для подсчёта общего числа установленных точек и числа точек вне области с отправкой оповещения при выходе за границы

Класс реализует MBean интерфейс для учёта всех точек и точек, вышедших за границы области, с генерацией JMX-уведомления при превышении порогового значения.

Листинг 1: PointsCounterMBean.java

```
1 package ru.itmo.web.lab3.beans;
2
3 import ru.itmo.web.lab3.model.Point;
4
5 public interface PointsCounterMBean {
6     void proceedPoint(Point point);
7     int getPointsCounter();
8     int getHitsCounter();
9     void resetCounters();
10 }
```

Листинг 2: PointsCounter.java

```
1 package ru.itmo.web.lab3.beans;
2
3 import jakarta.enterprise.context.ApplicationScoped;
4 import lombok.Setter;
5 import ru.itmo.web.lab3.model.Point;
6
7 import javax.management.Notification;
8 import javax.management.NotificationBroadcasterSupport;
9 import javax.management.ObjectName;
10 import java.io.Serializable;
11
12 @ApplicationScoped
13 public class PointsCounter extends NotificationBroadcasterSupport
14 implements PointsCounterMBean, Serializable {
15     private int pointsCounter = 0;
16     private int hitsCounter = 0;
17     private long notificationSequence = 0;
18     @Setter
19     private ObjectName objectName;
20     @Override
21     public void proceedPoint(Point point) {
22         if(point.isHit()) {
23             hitsCounter++;
24         }
25         pointsCounter++;
26         checkTenPoints();
27     }
28
29     @Override
30     public int getPointsCounter() {
31         return pointsCounter;
32     }
33
34     @Override
35     public int getHitsCounter() {
36         return hitsCounter;
37     }
38
39     @Override
40     public void resetCounters() {
41         pointsCounter = 0;
42         hitsCounter = 0;
43     }
44
45     void checkTenPoints() {
46         if(pointsCounter % 10 == 0) {
```

```
47         Notification notification = new Notification(  
48             "PointsCounter_Notification",  
49             this,  
50             notificationSequence++,  
51             System.currentTimeMillis(),  
52             "Reached_" + pointsCounter + "_points");  
53         sendNotification(notification);  
54     }  
55 }  
56 }
```

2.2 MBean для определения среднего интервала между кликами пользователя

Класс предназначен для измерения и усреднения времени между последовательными событиями клика.

Листинг 3: AverageClickTimeMBean.java

```
1 package ru.itmo.web.lab3.beans;
2
3 import ru.itmo.web.lab3.model.Point;
4
5 public interface AverageClickTimeMBean {
6     long getAverageClickTime();
7     void reset();
8     void proceedPoint(Point point);
9 }
```

Листинг 4: AverageClickTime.java

```
1 package ru.itmo.web.lab3.beans;
2
3 import jakarta.enterprise.context.ApplicationScoped;
4 import lombok.Setter;
5 import lombok.extern.log4j.Log4j2;
6 import ru.itmo.web.lab3.model.Point;
7
8 import javax.management.MBeanServer;
9 import javax.management.ObjectName;
10 import java.lang.management.ManagementFactory;
11 import java.time.Duration;
12 import java.time.LocalDateTime;
13
14 @ApplicationScoped
15 @Log4j2
16 public class AverageClickTime implements AverageClickTimeMBean {
17     @Setter
18     private ObjectName objectName;
19     @Setter
20     private ObjectName pointsCounterObjectName;
21     private LocalDateTime firstClickTime;
22     private LocalDateTime lastClickTime;
23     private int pointsCount;
24     private final MBeanServer mBeanServer;
25     private long averageClickTime;
26
27     public AverageClickTime() {
28         this.mBeanServer = ManagementFactory.getPlatformMBeanServer();
29         this.firstClickTime = null;
30         this.lastClickTime = null;
31         this.pointsCount = 0;
32     }
33
34     @Override
35     public long getAverageClickTime() {
36         return averageClickTime;
37     }
38
39     @Override
40     public void proceedPoint(Point point) {
41         if (point == null || point.getCheckTime() == null) {
42             return;
43         }
44
45         if (firstClickTime == null) {
46             firstClickTime = point.getCheckTime();
47         }
48
49         lastClickTime = point.getCheckTime();
50
51         try {
52             pointsCount = (Integer) mBeanServer.getAttribute(pointsCounterObjectName, "PointsCount");
53             log.info("Number of points in first click: {}", pointsCount);
54         } catch (Exception e) {
55             log.error("Error getting points count: {}", e.getMessage());
56         }
57     }
58 }
```

```

54     } catch (Exception e) {
55         log.error("Failed to get PointsCounter", e);
56         pointsCount = 0;
57     }
58
59     if (firstClickTime == null || lastClickTime == null || pointsCount <= 1) {
60         averageClickTime = -1;
61         log.info("Average click time is incorrect");
62         log.info("firstClickTime:{}", firstClickTime);
63         log.info("lastClickTime:{}", lastClickTime);
64         log.info("pointsCount:{}", pointsCount);
65         return;
66     }
67
68     try {
69         long durationMillis = Duration.between(firstClickTime, lastClickTime).toMillis();
70         averageClickTime = durationMillis / (pointsCount - 1);
71         log.info("New average click time:{}", averageClickTime);
72     } catch (Exception e) {
73         e.printStackTrace();
74         averageClickTime = -500;
75     }
76 }
77
78 @Override
79 public void reset() {
80     firstClickTime = null;
81     lastClickTime = null;
82     pointsCount = 0;
83 }
84 }

```

3 Мониторинг с помощью JConsole

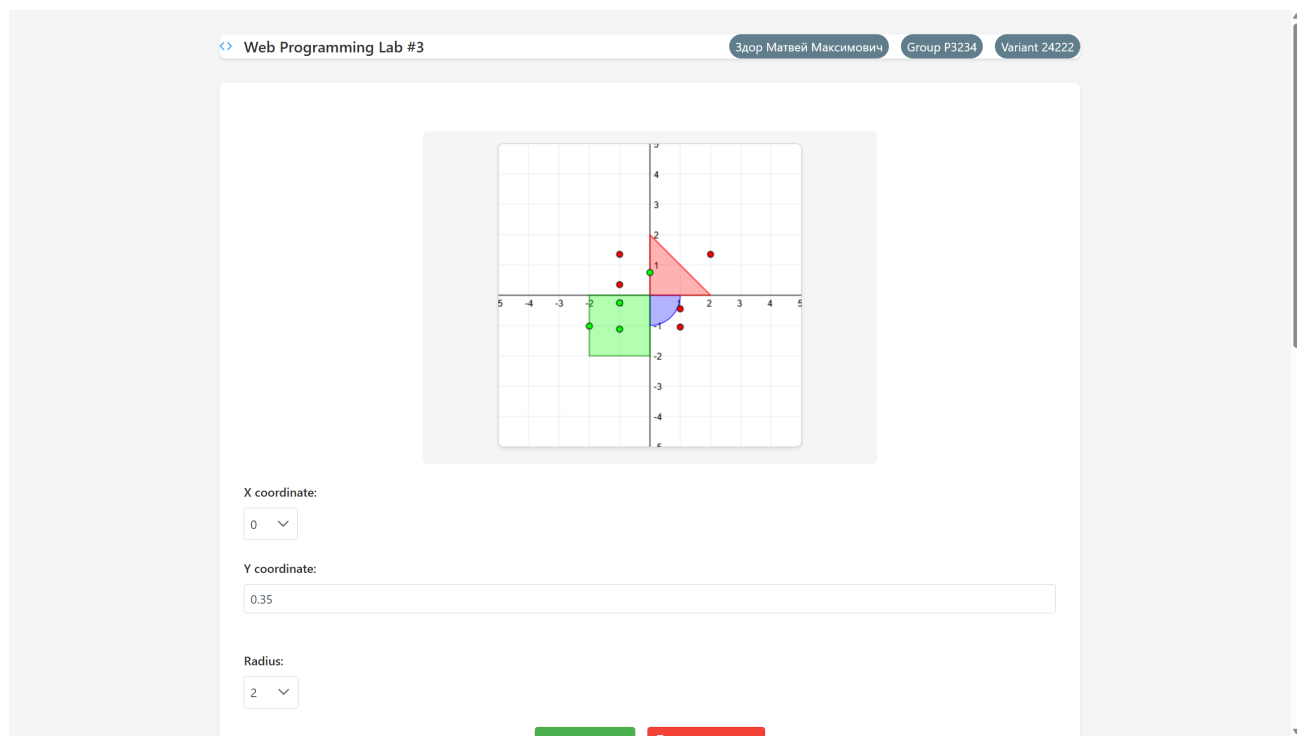


Рис. 2: Интерфейс веб-приложения с координатной плоскостью для установки точек

3.1 Снятие показаний MBean-классов

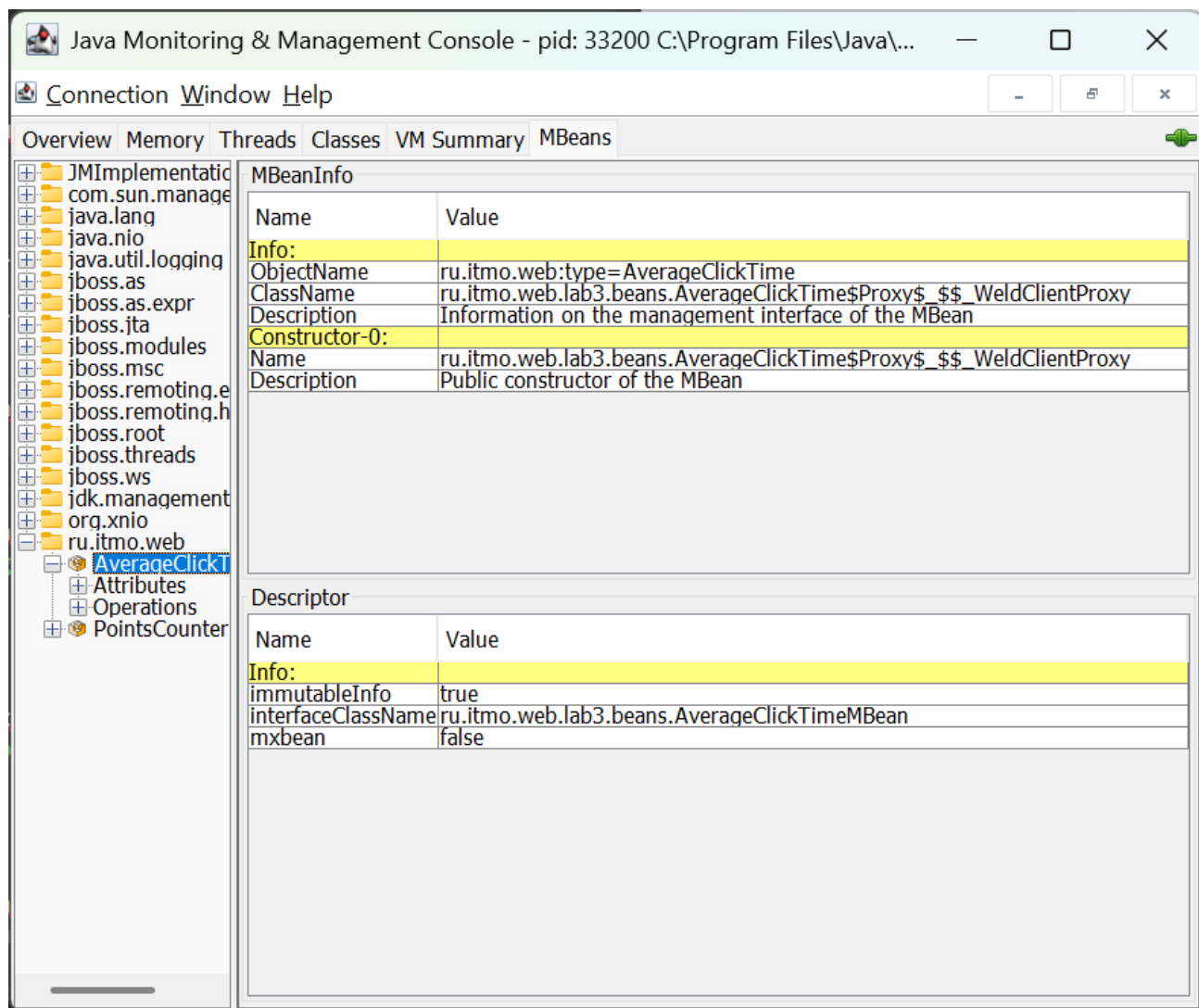


Рис. 3: Просмотр MBean AverageClickTime в JConsole

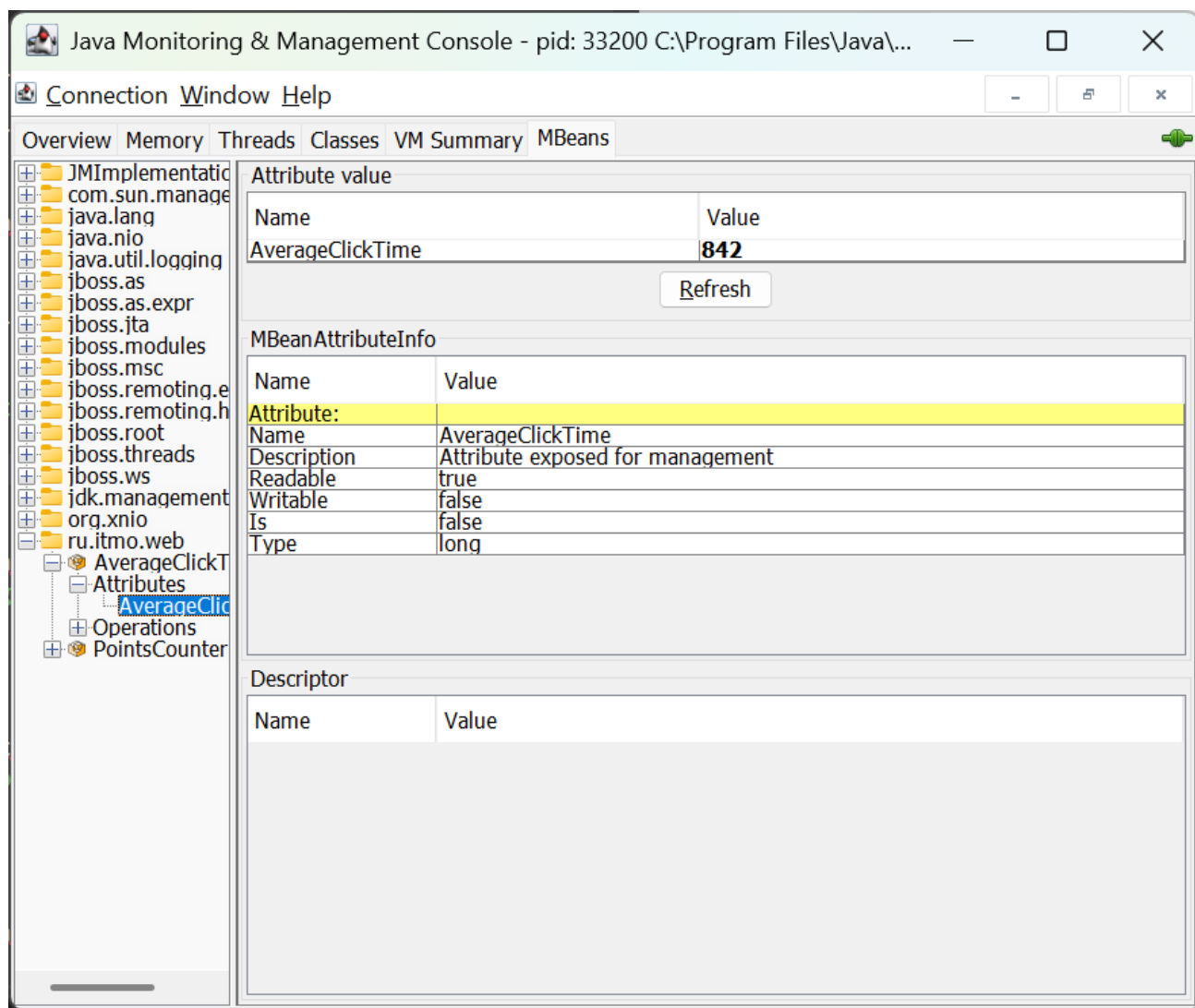


Рис. 4: Атрибуты MBean AverageClickTime, отображающие среднее время между кликами

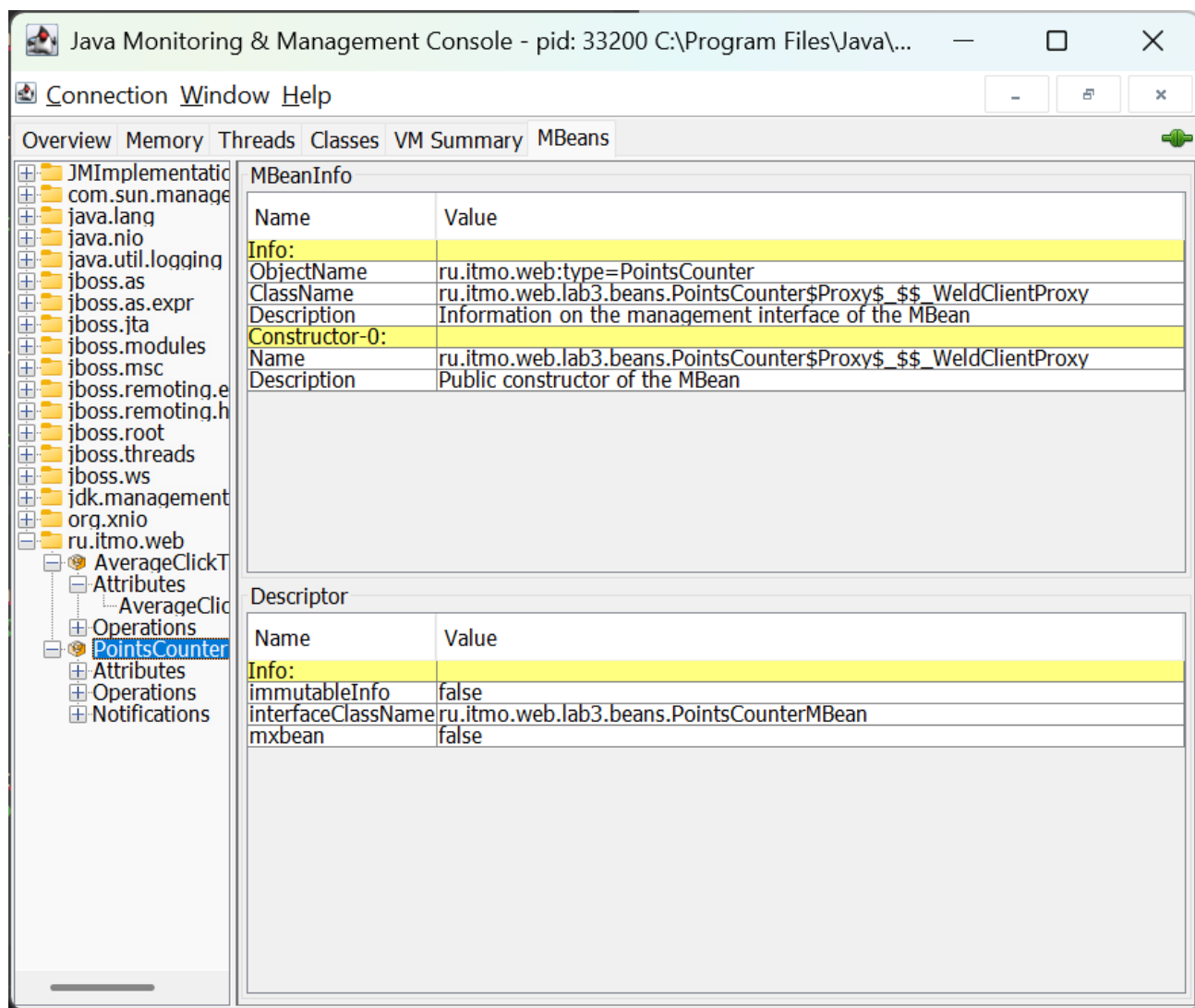


Рис. 5: Просмотр MBean PointsCounter в JConsole

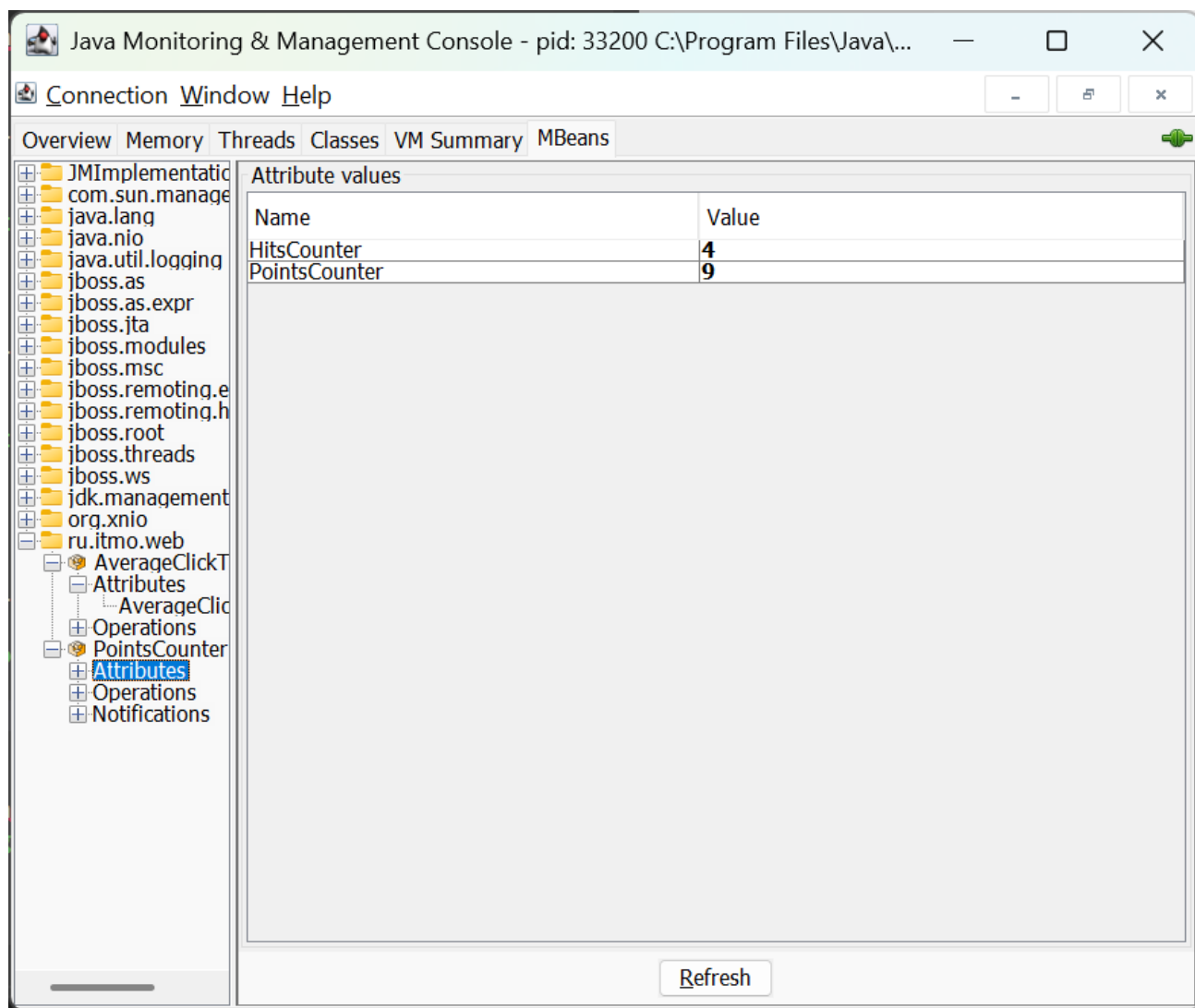


Рис. 6: Атрибуты MBean PointsCounter: общее число точек и число попаданий

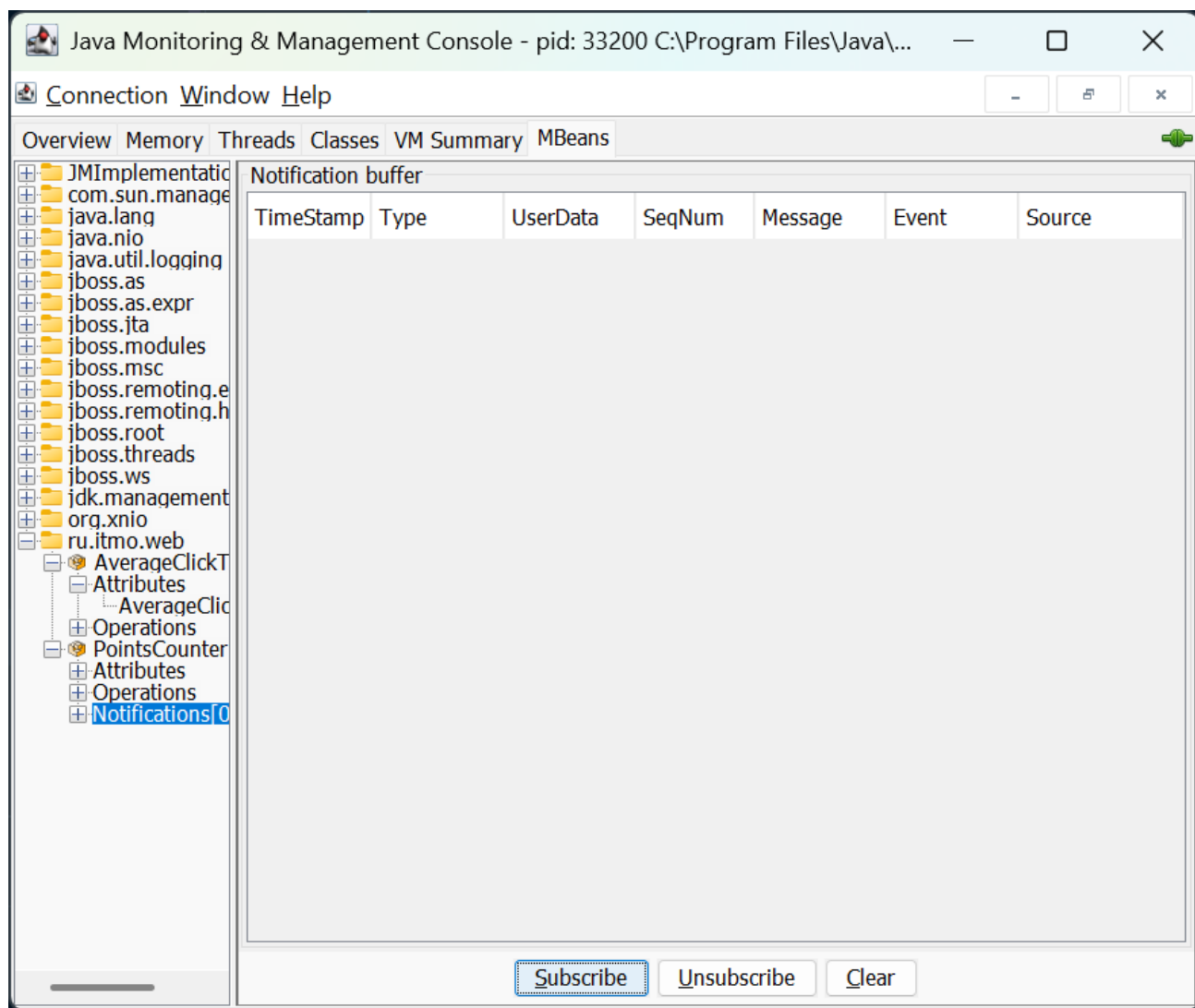


Рис. 7: Подписка на уведомления от PointsCounter в JConsole

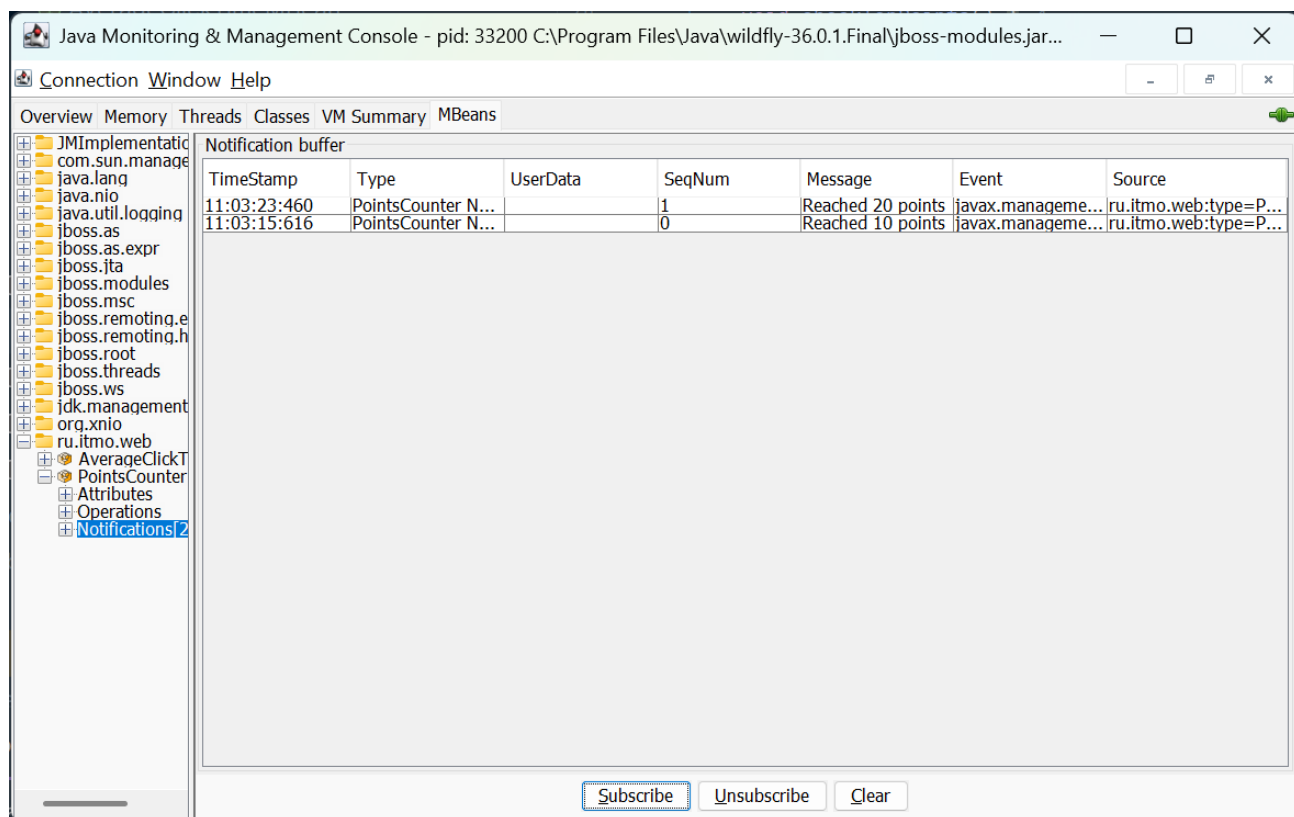


Рис. 8: Уведомления от PointsCounter при достижении числа точек, кратного 10

3.2 Определение Java Language Specification

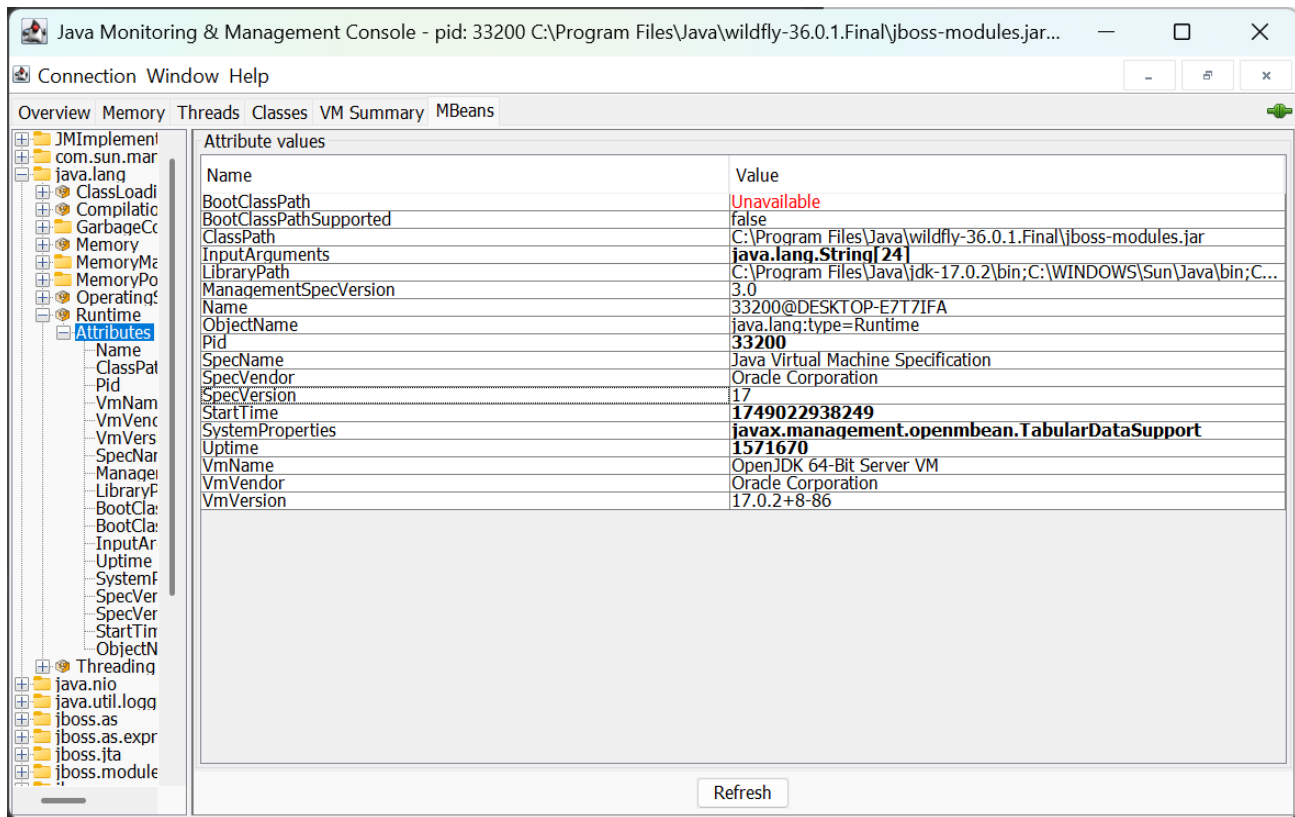


Рис. 9: Определение версии Java Language Specification в JConsole

4 Мониторинг и профилирование с помощью VisualVM

4.1 Графики изменения показаний MBean-классов во времени

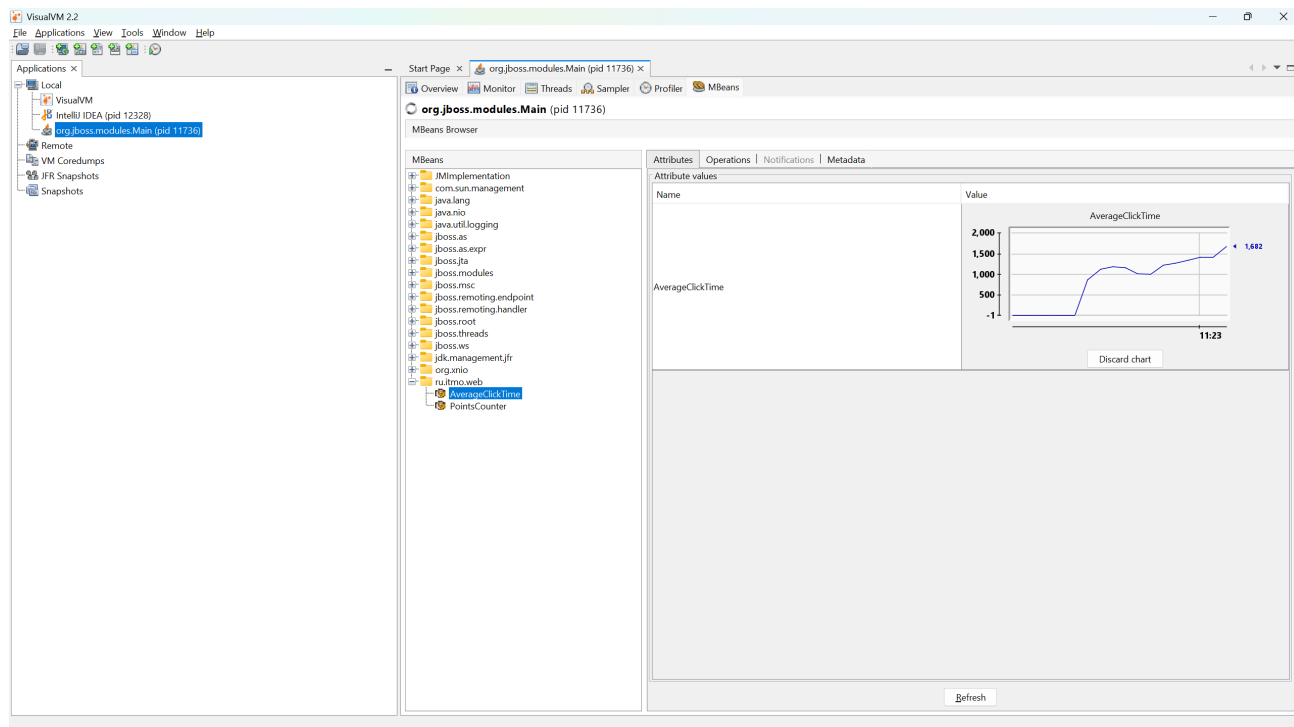


Рис. 10: График изменения среднего времени между кликами в VisualVM

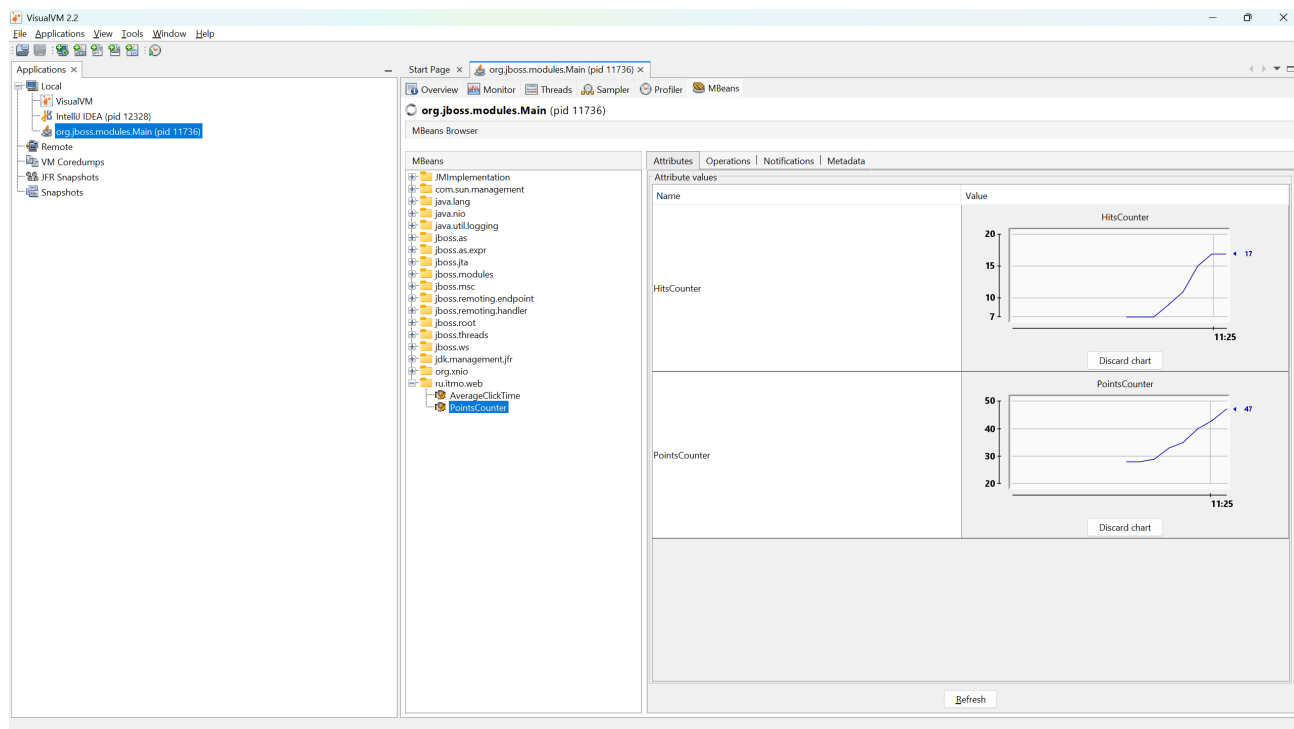


Рис. 11: График изменения числа точек и попаданий в VisualVM

4.2 Определение класса с наибольшим использованием памяти

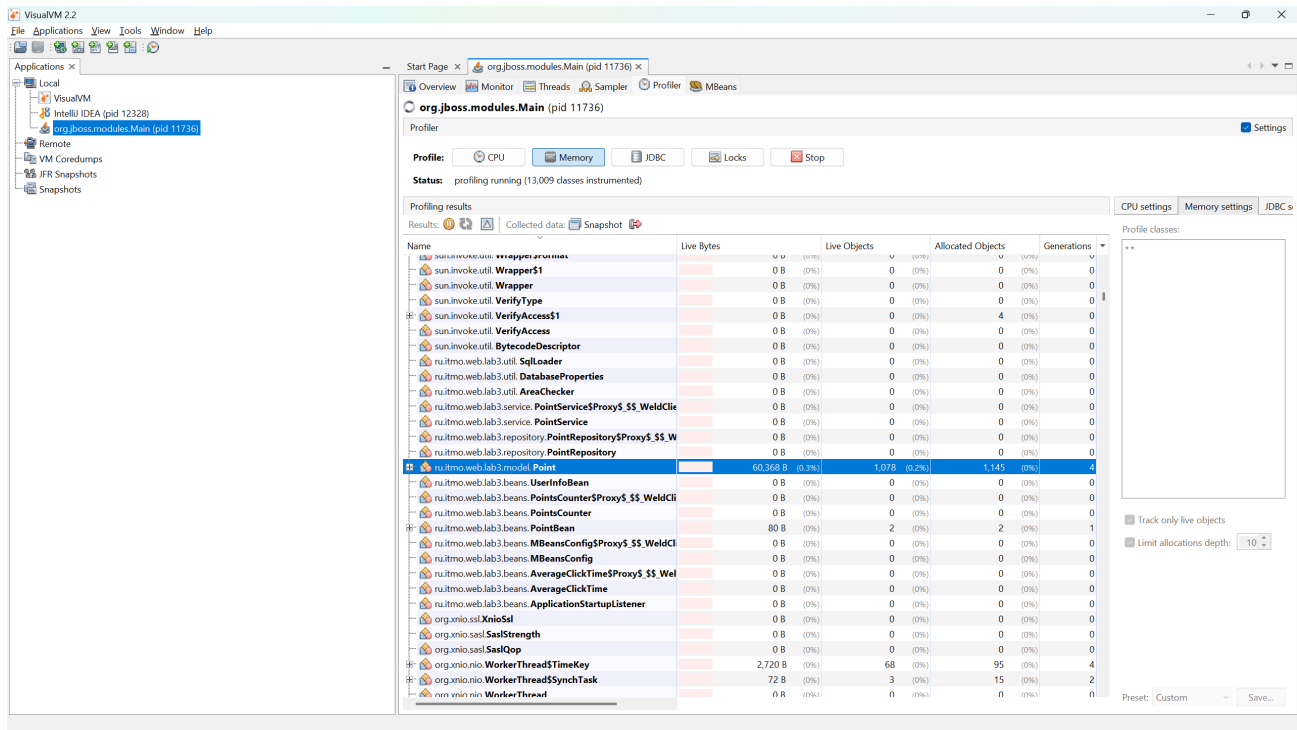


Рис. 12: Класс Point, занимающий наибольший объем памяти в куче

ru.itmo.web.lab3.model.Point	47,040 B (0.2%)	840 (0.1%)	2,044 (0%)
java.lang.Object.<init>()	47,040 B (0.2%)	840 (0.1%)	2,044 (0%)
ru.itmo.web.lab3.model.Point.<init>()	47,040 B (0.2%)	840 (0.1%)	2,044 (0%)
ru.itmo.web.lab3.service.PointService.processPoint()	56 B (0%)	1 (0%)	4 (0%)
ru.itmo.web.lab3.service.PointServiceProxy\$\$_WeldClient	56 B (0%)	1 (0%)	4 (0%)
ru.itmo.web.lab3.beans.PointBean.checkPoint()	56 B (0%)	1 (0%)	4 (0%)
jdk.internal.reflect.NativeMethodAccess	56 B (0%)	1 (0%)	4 (0%)
jdk.internal.reflect.NativeMethodAccess	56 B (0%)	1 (0%)	4 (0%)
jdk.internal.reflect.DelegatingMethodAccessor	56 B (0%)	1 (0%)	4 (0%)
java.lang.reflect.Method.invoke()	56 B (0%)	1 (0%)	4 (0%)
org.glassfish.expressly.util.ReflectionUtil	56 B (0%)	1 (0%)	4 (0%)
ru.itmo.web.lab3.repository.PointRepository.findPoint()	46,984 B (0.2%)	839 (0.1%)	2,040 (0%)
ru.itmo.web.lab3.repository.PointRepository\$FindPoint	46,984 B (0.2%)	839 (0.1%)	2,040 (0%)
ru.itmo.web.lab3.service.PointService.getPoint()	46,984 B (0.2%)	839 (0.1%)	2,040 (0%)
ru.itmo.web.lab3.service.PointService\$GetPoint	46,984 B (0.2%)	839 (0.1%)	2,040 (0%)
ru.itmo.web.lab3.beans.PointBean.getPoints()	46,984 B (0.2%)	839 (0.1%)	2,040 (0%)
jdk.internal.reflect.GeneratedMethodAccessor	46,984 B (0.2%)	839 (0.1%)	2,040 (0%)
jdk.internal.reflect.DelegatingMethodAccessor	46,984 B (0.2%)	839 (0.1%)	2,040 (0%)
java.lang.reflect.Method	46,984 B (0.2%)	839 (0.1%)	2,040 (0%)

Рис. 13: Класс, содержащий объекты Point (обращение к базе данных)


```

39
40 /**
41  * Получает список всех точек.
42  *
43  * @return список точек
44  */
45 public List<Point> getPoints() { return pointService.getAllPoints(); }

```

Рис. 14: Метод, загружающий все точки из базы данных

5 Локализация и устранение проблем производительности

Запустим приложение и откроем в visualvm

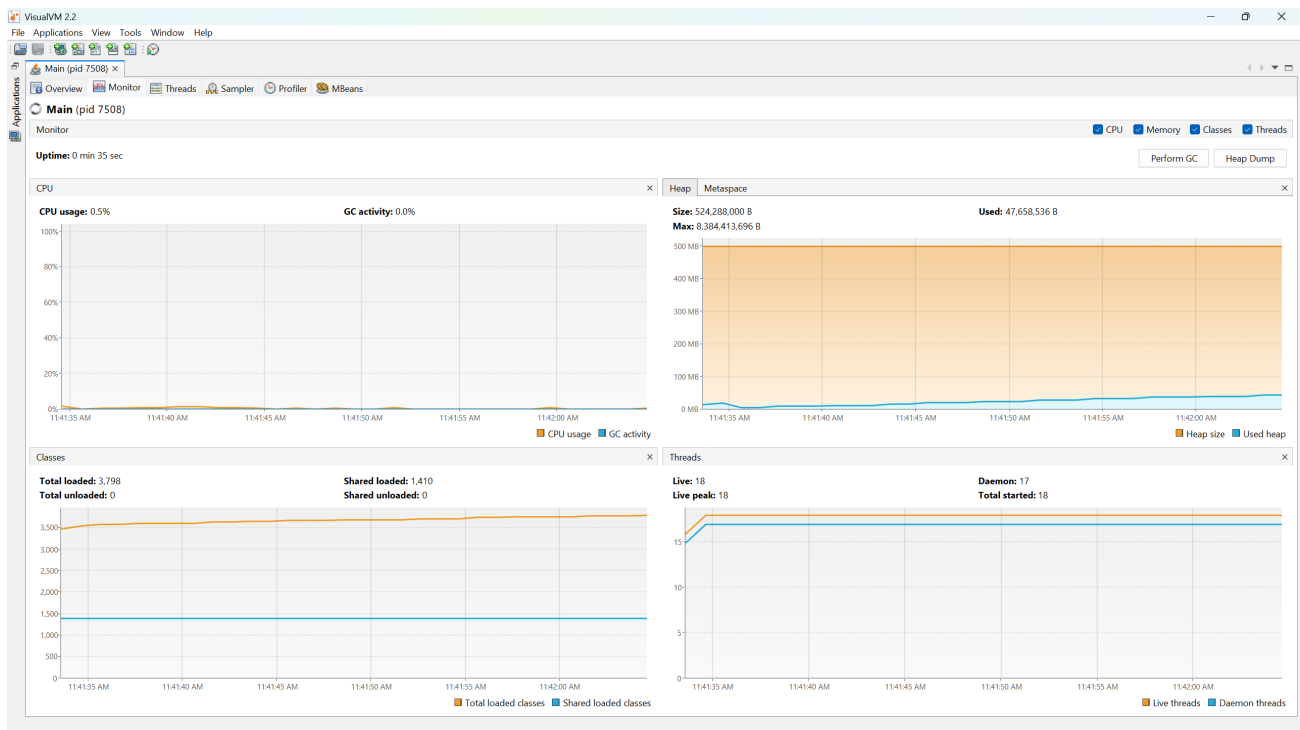


Рис. 15: Запуск приложения в VisualVM для мониторинга

Ограничим память и уберём Thread.sleep, чтобы ускорить отправку запросов и тестирование

```

42 ServletUnitClient sc = sr.newClient();
43 int number = 1;
44 WebRequest request = new GetMethodWebRequest( urlString: "http://test.meterware.
45 while (true) {
46     WebResponse response = sc.getResponse(request);
47     System.out.println("Count: " + number++ + response);
48     // java.lang.Thread.sleep(200);
49
50 }
51 // catch (InterruptedException ex) {
52 //     Logger.getLogger("global").log(Level.SEVERE, null, ex);
53 // }
54 catch (MalformedURLException ex) {

```

Рис. 16: Удаление Thread.sleep для ускорения тестирования

Name: ☐ Store as project file

Run on:

Run configurations may be executed locally or on a target: for example in a Docker Container or on a remote host using SSH.

Build and run [Modify options](#) Alt+M

Press Alt for field hints

Рис. 17: Ограничение памяти JVM до 20 МБ с параметром -Xmx20m

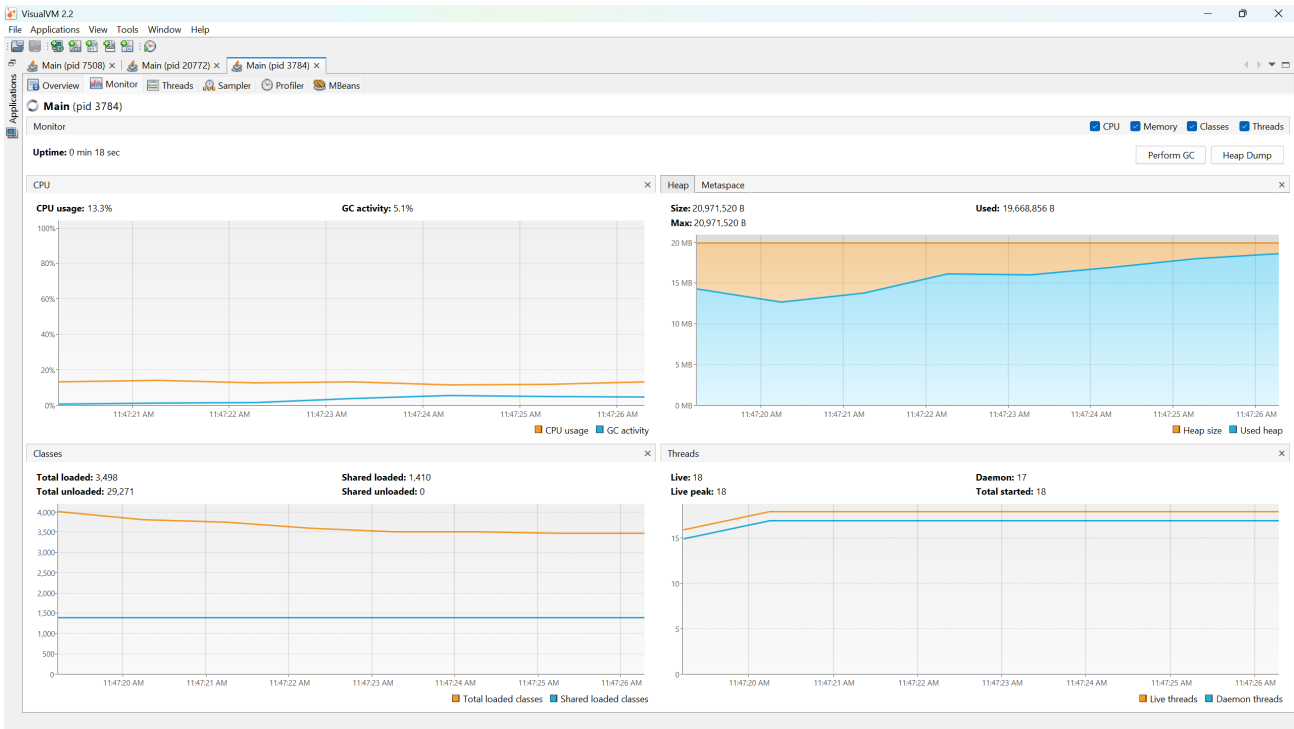


Рис. 18: Рост использования кучи до завершения программы за 20 секунд

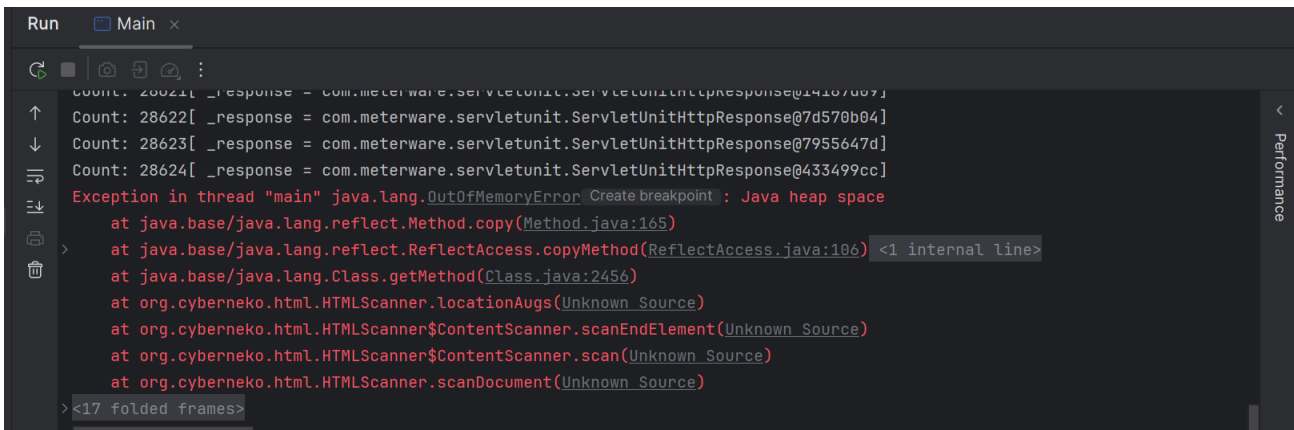


Рис. 19: Ошибка OutOfMemoryError при исчерпании памяти

Увеличим память до 25 и сделаем heap dump когда heap накопится, делаем ещё несколько heap dump через некоторое время

Heap	
Size:	21,259,024 B
Classes:	3,862
Instances:	523,284
Classloaders:	10
GC Roots:	3,407
Objects Pending for Finalization:	0

Рис. 20: Heap dump 1: начальное состояние кучи при увеличении памяти до 25 МБ

Heap	
Size:	22,209,560 B
Classes:	3,863
Instances:	533,581
Classloaders:	10
GC Roots:	3,396
Objects Pending for Finalization:	0

Рис. 21: Heap dump 2: промежуточное состояние кучи через несколько секунд

Heap	
Size:	23,843,008 B
Classes:	3,872
Instances:	553,651
Classloaders:	11
GC Roots:	3,401
Objects Pending for Finalization:	0

Рис. 22: Heap dump 3: финальное состояние кучи с накоплением объектов

Замечаем увеличение количества инстансов со временем, открываем последний дамп и смотрим какие объекты создаются

Name	Count	Size	Retained (sort to get)
byte[]	78,914 (14.3%)	8,069,984 B (33.8%)	n/a
java.util.TreeMap\$Entry	107,240 (19.4%)	4,289,600 B (18%)	n/a
java.lang.String	78,004 (14.1%)	1,872,096 B (7.9%)	n/a
java.util.HashMap	34,045 (6.1%)	1,634,160 B (6.9%)	n/a
java.lang.Long	50,526 (9.1%)	1,212,624 B (5.1%)	n/a
java.lang.Object	20,597 (3.7%)	924,664 B (3.9%)	n/a
java.util.HashMap\$Entry	34,000 (6.1%)	816,000 B (3.4%)	n/a
java.util.HashMap\$KeySet	17,204 (3.1%)	688,160 B (2.9%)	n/a
int[]	34,012 (6.1%)	544,192 B (2.3%)	n/a
java.util.Collections\$UnmodifiableRandomAccessList	1,075 (0.2%)	511,360 B (2.1%)	n/a
java.util.ArrayList	16,280 (2.9%)	390,720 B (1.6%)	n/a
java.util.HashMap\$Node	16,049 (2.9%)	385,176 B (1.6%)	n/a
java.util.concurrent.ConcurrentHashMap\$Node	2,912 (0.5%)	247,800 B (1%)	n/a
java.util.HashMap\$Node	5,705 (1%)	182,560 B (0.8%)	n/a
java.util.concurrent.ConcurrentHashMap\$Node	2,433 (0.4%)	155,712 B (0.7%)	n/a
java.util.HashMap\$Node	3,659 (0.7%)	117,088 B (0.5%)	n/a
java.lang.reflect.Method	1,236 (0.2%)	108,768 B (0.5%)	n/a
char[]	256 (0%)	103,000 B (0.4%)	n/a
java.lang.String[]	3,245 (0.6%)	89,680 B (0.4%)	n/a
java.util.ImmutableCollections\$List12	2,353 (0.4%)	56,472 B (0.2%)	n/a
java.util.concurrent.ConcurrentHashMap\$Node	106 (0%)	52,448 B (0.2%)	n/a
java.management.ObjectName\$Property	2,063 (0.4%)	49,504 B (0.2%)	n/a
java.management.ObjectName\$Property	2,050 (0.4%)	49,200 B (0.2%)	n/a
java.management.openmbean.TabularDataSupport	2,000 (0.4%)	48,000 B (0.2%)	n/a
java.management.Notification	1,000 (0.2%)	48,000 B (0.2%)	n/a
java.lang.invoke.MemberName	1,175 (0.2%)	47,000 B (0.2%)	n/a
java.lang.invoke.AnnotationElement	1,572 (0.3%)	37,728 B (0.2%)	n/a
java.lang.Class	1,311 (0.2%)	36,736 B (0.2%)	n/a
java.lang.invoke.AnnotationConstruct	1,481 (0.3%)	35,544 B (0.1%)	n/a
java.lang.invoke.MethodType	879 (0.2%)	35,160 B (0.1%)	n/a
java.management.ObjectName	1,032 (0.2%)	33,024 B (0.1%)	n/a
java.lang.invoke.ValueDescriptor	806 (0.1%)	32,240 B (0.1%)	n/a

Рис. 23: Анализ объектов в последнем heap dump

Недолго просмотрев объекты занимающие больше всего памяти замечаем повторяющиеся ошибки

Name	Count	Size	Retained (sort to get)
byte[]	78,914 (14.3%)	8,069,984 B (33.8%)	n/a
java.util.TreeMap\$Entry	107,240 (19.4%)	4,289,600 B (18%)	n/a
java.lang.String	78,004 (14.1%)	1,872,096 B (7.9%)	n/a
java.util.TreeMap	34,045 (6.1%)	1,634,160 B (6.9%)	n/a
java.lang.Long	50,526 (9.1%)	1,212,624 B (5.1%)	n/a
java.lang.Object	20,597 (3.7%)	924,664 B (3.9%)	n/a
java.lang.Object[]		284,576 B (1.2%)	n/a

Рис. 24: Обнаружение накопления ошибок в heap dump

Смотрим класс их хранения

Name	Count	Size	Retained (sort to get)
byte[]	78,914 (14.3%)	8,069,984 B (33.8%)	n/a
java.util.TreeMap\$Entry	107,240 (19.4%)	4,289,600 B (18%)	n/a
java.lang.String	78,004 (14.1%)	1,872,096 B (7.9%)	n/a
java.util.TreeMap	34,045 (6.1%)	1,634,160 B (6.9%)	n/a
java.lang.Long	50,526 (9.1%)	1,212,624 B (5.1%)	n/a
java.lang.Object	20,597 (3.7%)	924,664 B (3.9%)	n/a
java.lang.Object[]		284,576 B (1.2%)	n/a

Рис. 25: Локализация ошибок в ArrayList класса JavaScript

ArrayList хранит все эти ошибки и находится в классе JavaScript, заходим в него и смотрим поля класса

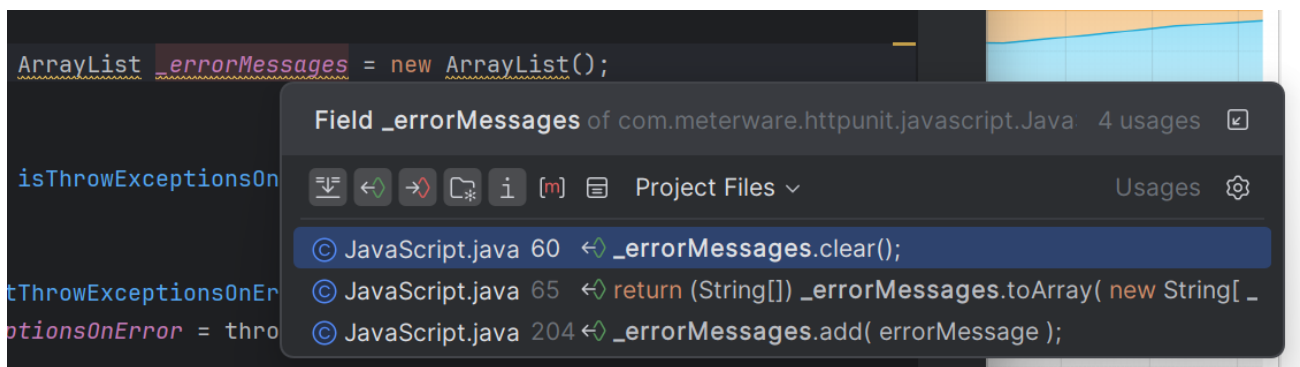
```

41
42     private final static Object[] NO_ARGS = new Object[0];
43
44     private static boolean _throwExceptionsOnError = true;
45
46     private static ArrayList _errorMessages = new ArrayList();
47
48
49     > static boolean isThrowExceptionsOnError() { return _throwExceptionsOnError; }
52

```

Рис. 26: Поля класса JavaScript, содержащие ArrayList

Видим только 1 ArrayList, смотрим где он дополняется и очищается



```

ArrayList _errorMessages = new ArrayList();

isThrowExceptionsOn

tThrowExceptionsOnEr
ptionsOnError = thro

Field _errorMessages of com.meterware.httpunit.javascript.Java: 4 usages
[Download] [Copy] [Paste] [Find] [Project Files] [Usages] [Settings]
© JavaScript.java 60 ↔ _errorMessages.clear();
© JavaScript.java 65 ↔ return (String[]) _errorMessages.toArray( new String[ _
© JavaScript.java 204 ↔ _errorMessages.add( errorMessage );

```

Рис. 27: Методы добавления элементов в ArrayList

```

/**
 * Clears the accumulated script error messages.
 */
public static void clearScriptErrorMessages() { no usage
    getScriptingEngine().clearErrorMessages();
}

```

Рис. 28: Метод очистки ArrayList, который не вызывался

Обнаруживаем, что очистка не вызывается в коде, добавляем самое простое решение, очистку в цикле запросов Main

```

39 // catch (InterruptedException ex) {
40 //     Logger.getLogger("global").log(Level.SEVERE, null, ex);
41 // }
42 //
43 // while (true) {
44 //     ServletRunner sr = new ServletRunner();
45 //     sr.registerServlet("myServlet", HelloWorld.class.getName());
46 //     ServletUnitClient sc = sr.newClient();
47 //     int number = 1;
48 //     WebRequest request = new GetMethodWebRequest("http://test.meterware.");
49 //     while (true) {
50 //         WebResponse response = sc.getResponse(request);
51 //         System.out.println("Count: " + number++ + response);
52 //         HttpUnitOptions.clearScriptErrorMessages();
53 //         java.lang.Thread.sleep(200);
54 //     }
55 // }

```

Рис. 29: Добавление очистки ArrayList в цикл запросов

Снова запускаем с кучей размера 20 мб

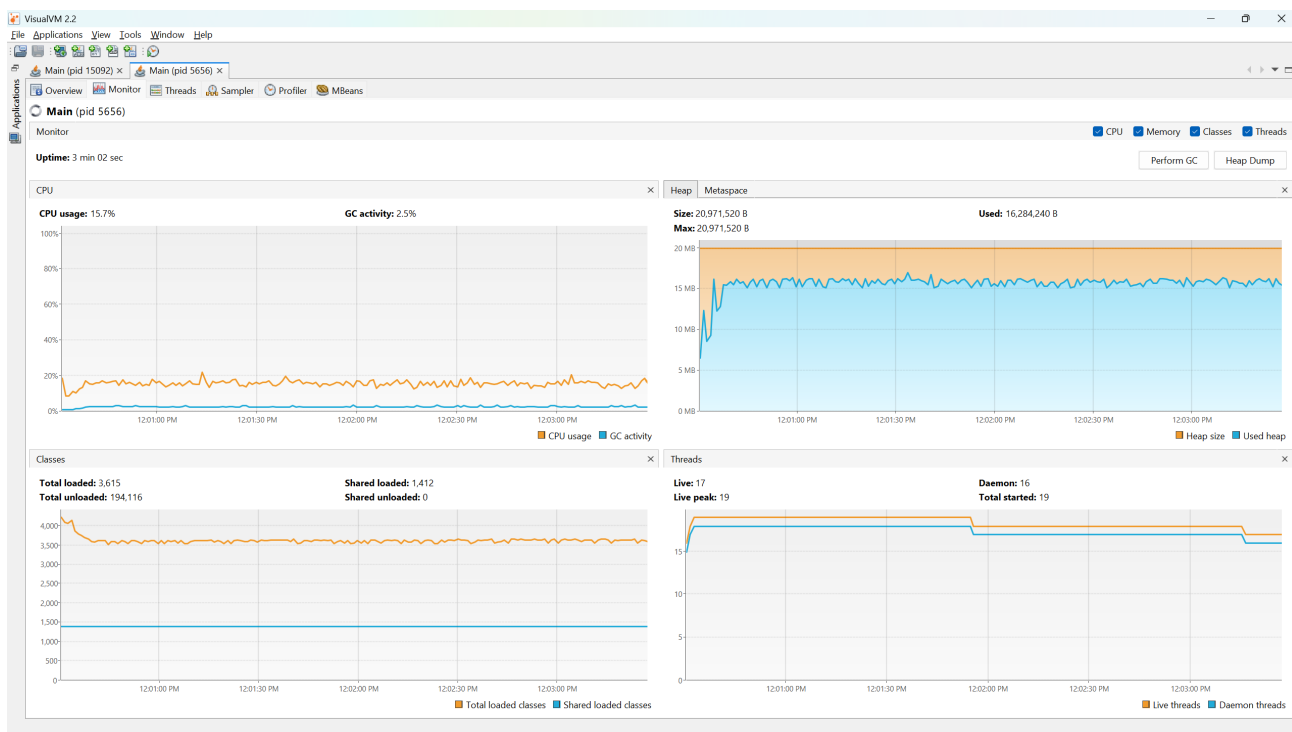


Рис. 30: Стабильная работа приложения с ограничением памяти 20 МБ после исправления

Программа стабильно работает через три минуты после запуска (при первом запуске завершилась через 20 секунд)

6 Заключение и выводы

Результаты:

- Реализованы MBean: `PointsCounter` (подсчет точек с уведомлениями каждые 10) и `AverageClickTime` (расчет среднего времени кликов).
- Проведен мониторинг в JConsole и профилирование в VisualVM: определена версия Java, построены графики, выявлен класс `Point` как основной потребитель памяти.
- Устранена утечка памяти в `ArrayList` путем очистки списка.

Выводы:

- MBean созданы, мониторинг и оптимизация проведены.
- Процесс поиска утечки памяти задокументирован.